

ASSIGNMENT 2

Natural Language Processing

SUBJECT CODE : SECJ5043-15

SUBJECT TITLE : KEPINTARAN BUATAN LANJUTAN (ADVANCED ARTIFICIAL INTELLIGENCE)

PROGRAMME : BACHELOR OF COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH HONOURS

STUDENT'S NAME : RASHADUL NAFIS RIYAD

MATRIC NO. : A22MJ3010

COLAB LINK : [Google Colab](#)

Assignment 2: Natural Language Processing Report

Dataset: IMDB Dataset of 50K Movie Reviews (Kaggle: lakshmi25npathi/imdb-dataset-of-50k-movie-reviews)

Tasks covered: Text Classification, Text Clustering, and Sentiment Analysis

Q1. Task Definition and NLP Background

Text classification is the task of assigning a piece of text to one of a fixed set of predefined categories. A model is trained on examples that already carry the correct label, and once it has learned the patterns that separate the categories, it can label new, unseen text on its own. Because the model learns from labelled examples, this is a supervised learning task. In practice, text classification powers email spam filters that decide whether a message is spam or not, customer support systems that route an incoming ticket to the correct department, and news platforms that automatically tag an article as sport, politics, or technology.

Text clustering also groups text together, but it does so without ever seeing a label. Instead, the algorithm measures how similar documents are to one another based on the words they share, and groups the most similar documents together, allowing structure to emerge from the data itself. Because no labelled examples are used, this is an unsupervised learning task. It is commonly used to group large volumes of customer feedback into recurring themes, to organise unlabelled document archives by topic, and to discover natural segments within a body of text that would otherwise be too large to read manually.

Sentiment analysis examines a piece of text and determines the emotional tone or opinion it conveys, typically positive or negative. It can be viewed as a specialised form of classification, where the categories represent feelings or attitudes rather than general topics. Sentiment analysis is widely used to monitor public opinion of a brand on social media, to summarise the overall mood expressed across thousands of product or service reviews, and to track changes in customer satisfaction over time.

This report applies all three tasks to the same dataset of movie reviews: text classification and sentiment analysis both predict the positive or negative label attached to each review, while text clustering explores whether reviews form natural groupings in the absence of that label.

Q2. Dataset Selection and Preprocessing

Dataset Chosen and Justification

The dataset selected for this assignment is the IMDB Dataset of 50K Movie Reviews, a binary sentiment classification benchmark originally introduced by Maas et al. (ACL 2011) and made publicly available on Kaggle. The dataset contains 50,000 movie reviews, each labelled as either positive or negative based on the reviewer's original rating.

This dataset was chosen because it suits all three tasks required by the assignment. The sentiment label provides the ground truth target for the classification models in Q3 and the sentiment analysis models in Q5, while the same label can be used as a reference point when interpreting the unsupervised clusters formed in Q4. Using a single dataset across all three tasks also keeps the final cross-task comparison in Q6 consistent, since every model is working with the same underlying text. In addition, the dataset is large enough to train meaningful models while remaining manageable within the Google Colab environment, and it reflects realistic, unstructured review text that requires genuine cleaning before it can be used.

Exploratory Data Analysis

Before any cleaning was performed, the dataset was inspected to understand its structure and quality. The dataset initially contained 50,000 rows and two columns, review and sentiment, both stored as text. No missing values were found in either column. However, 418 duplicate reviews were identified and removed, reducing the dataset to 49,582 rows. The class distribution was found to be close to balanced, as shown in Table 2.1 and Figure 2.1.

Table 2.1: Distribution of positive and negative reviews after duplicate removal.

Sentiment	Number of Reviews
Positive	24,884
Negative	24,698

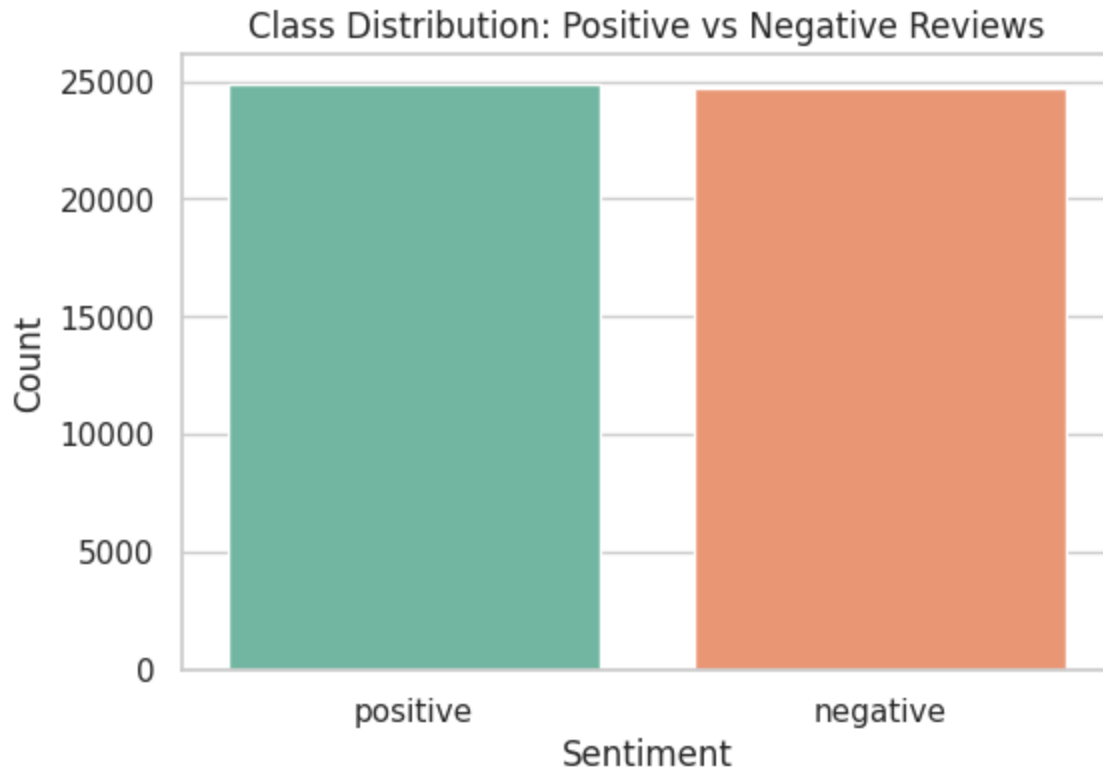


Figure 2.1: Bar chart of class distribution

Review length was also examined to understand the scale of the text being processed. Reviews varied considerably in length, averaging approximately 231 words with a median of 173 words, and ranging from as few as 4 words to as many as 2,470 words in a single review. This wide range indicated that no fixed-length assumption could be made about the raw text before vectorization.

A word frequency analysis was performed on the raw, uncleaned text to identify what the data actually contained before any preprocessing. The most frequent terms were dominated by common stopwords such as “the”, “a”, and “and”, which is expected. More importantly, the token “/><br” appeared 100,211 times among the top twenty most frequent terms — this is a remnant of HTML line-break tags (

) embedded directly in the raw review text. This finding was significant, as it confirmed that explicit HTML removal was required before tokenization, and that punctuation removal alone would not be sufficient to eliminate this artefact. Figure 2.2 illustrates the word cloud generated from raw text that is not being clean or preprocessed yet.

“film”, “one”, “like”, and “time”, as summarised in Table 2.2. Figure 2.3 shows the word cloud generated from text that has been cleaned and *lemmatized*.

Table 2.2: Comparison of the most frequent terms before and after preprocessing.

Rank	Top Words (Raw Text)	Top Words (Cleaned Text)
1	the	movie
2	a	film
3	and	one
4	of	like
5	to	time
6	is	good
7	/><br (HTML artefact)	character

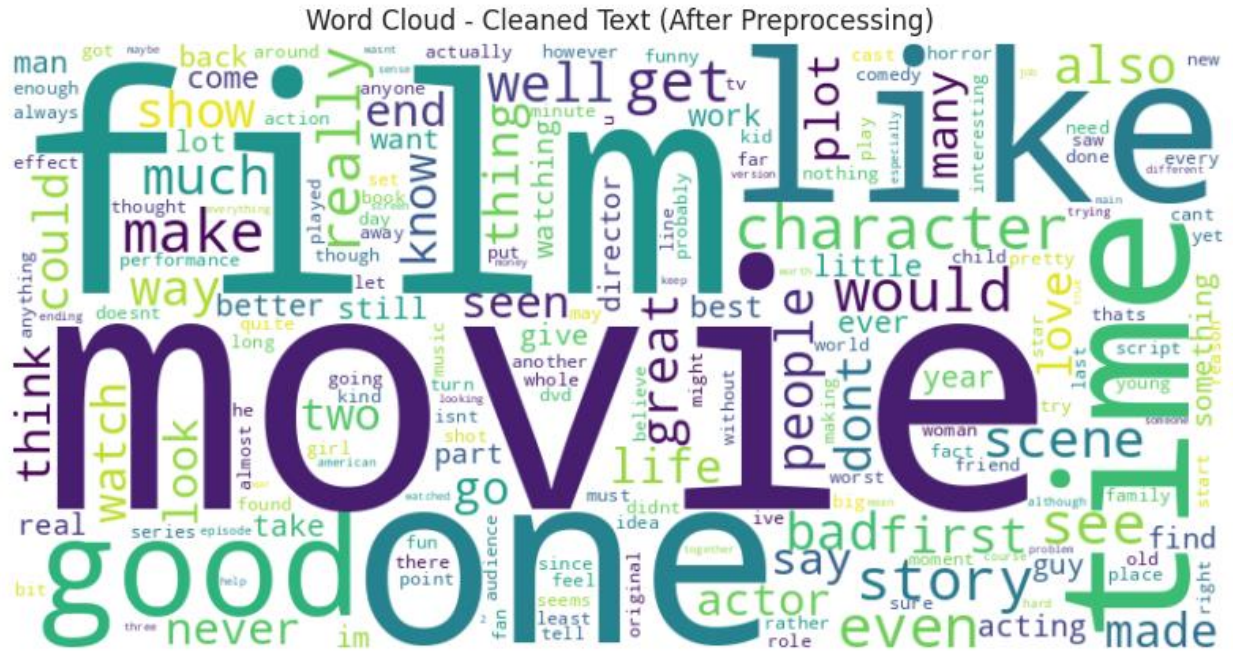


Figure 2.3: Word cloud generated from the cleaned, lemmatized review text.

Vectorization

Before vectorization, the dataset was split into training and testing sets using an 80/20 ratio (39,665 training reviews and 9,917 testing reviews), stratified by sentiment label with a fixed random seed of 42. This split was performed before vectorization specifically to avoid data leakage, since fitting a vectorizer on the full dataset — including the test set — would allow test-set vocabulary statistics to influence the training representation.

The cleaned text was then converted into numeric form using TF-IDF (Term Frequency – Inverse Document Frequency), which assigns each word a score that increases with its frequency in a given review but decreases the more commonly it appears across the entire corpus, so that distinctive words are emphasized over generic ones. TF-IDF was selected over embedding-based approaches because it integrates directly with the classical machine learning classifiers used in Q3, requires no pretrained models or additional setup, and keeps the overall pipeline simple and consistent with the rest of the implementation. The vectorizer was fitted on the training text only and then used to transform both the training and testing text, producing a training matrix of shape $(39,665 \times 136,532)$ and a testing matrix of shape $(9,917 \times 136,532)$.

Q3. Model Implementation for Text Classification

The goal of this section was to predict a review's sentiment label directly from its TF-IDF representation. Three classical classifiers were selected, each suited to high-dimensional, sparse text features:

- Multinomial Naive Bayes — a fast, probability-based model commonly used as a baseline for text classification, which assumes word occurrences are independent of one another.
- Logistic Regression — a linear model that learns a weight for every word indicating how strongly it contributes to a positive or negative prediction.
- Linear SVM — a margin-based linear model that searches for the boundary that most cleanly separates the two classes, and is known to perform well on high-dimensional sparse data such as TF-IDF.

Results

Table 3.1: Performance comparison of the three text classification models on the held-out test set.

Model	Accuracy	Precision	Recall	F1-Score
Naive Bayes	0.8666	0.8668	0.8666	0.8666
Logistic Regression	0.8925	0.8929	0.8925	0.8925
Linear SVM	0.8954	0.8955	0.8954	0.8954

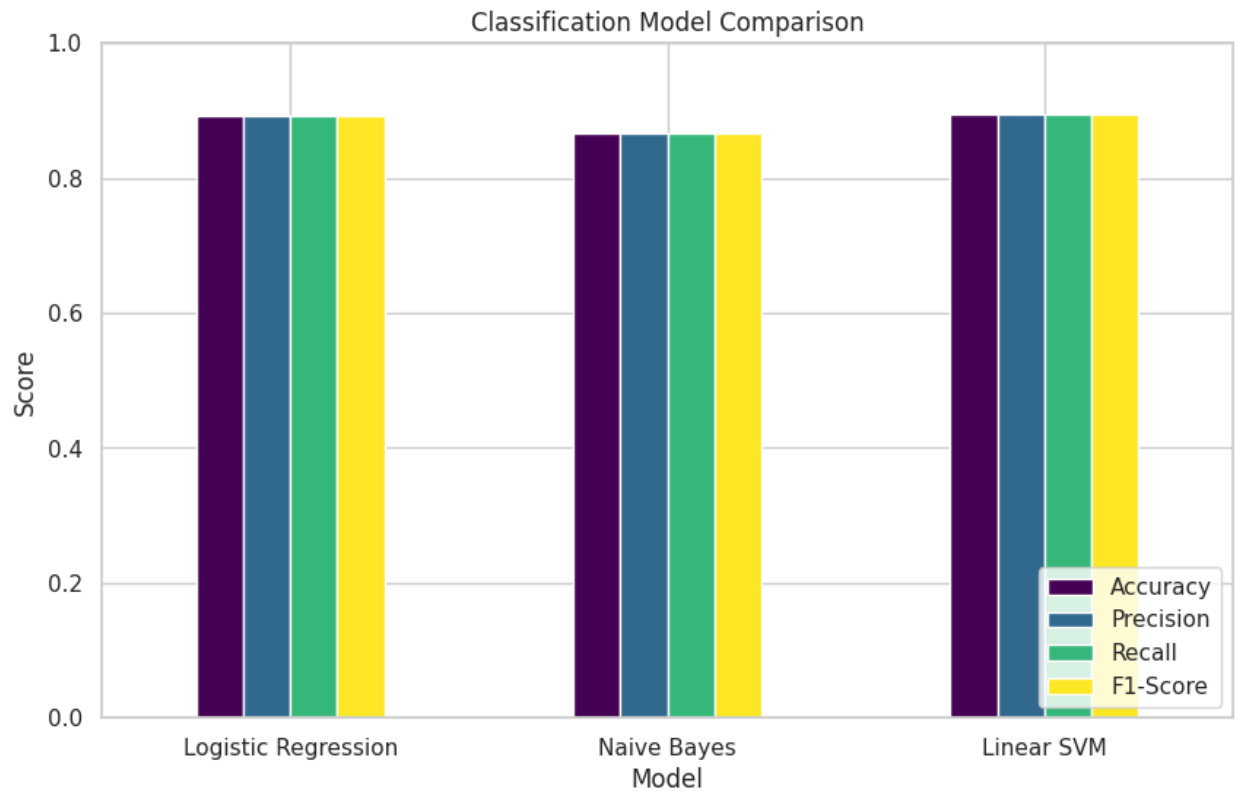


Figure 3.1: Bar chart comparing accuracy, precision, recall, and F1-score across the three classification models.

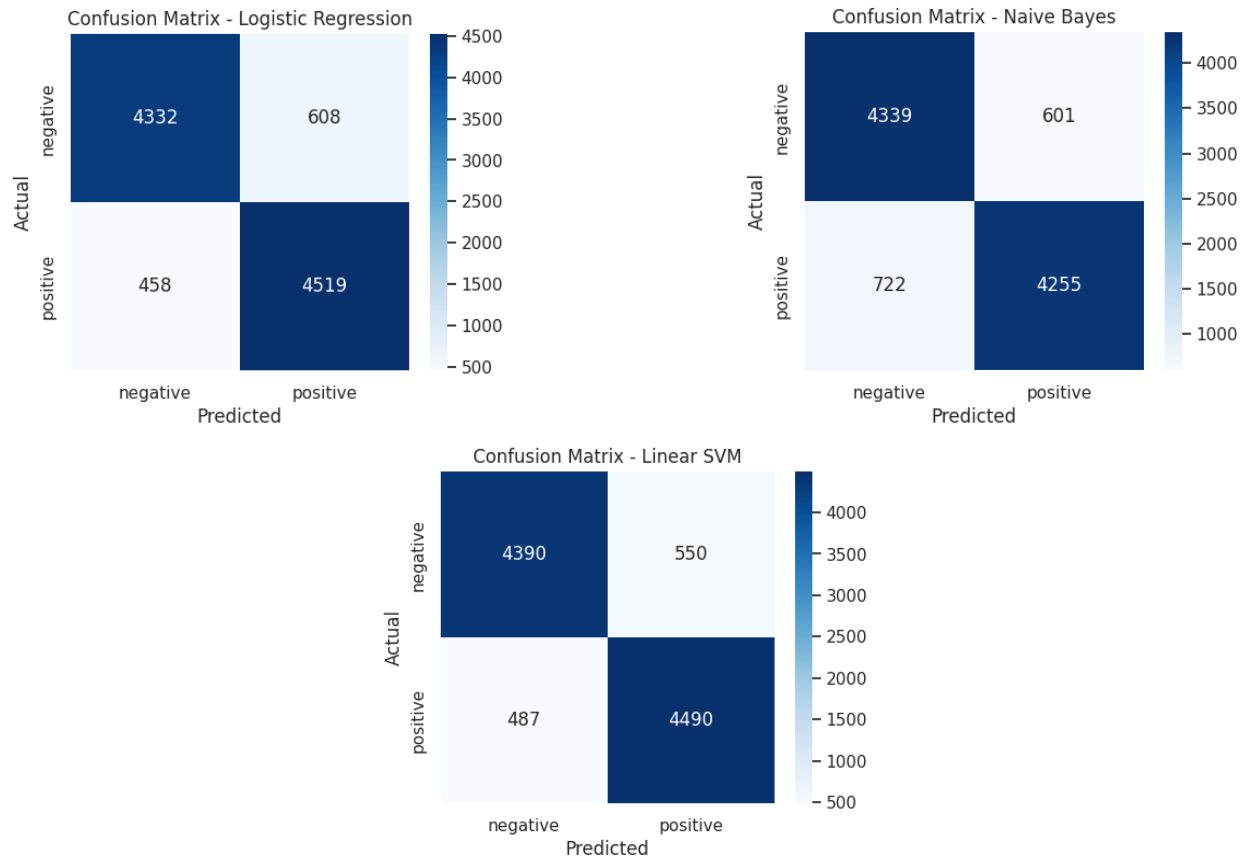


Figure 3.2: Confusion matrices for Logistic Regression, Naive Bayes, and Linear SVM.

Linear SVM achieved the highest accuracy (89.54%), narrowly ahead of Logistic Regression (89.25%), with Naive Bayes trailing both at 86.66%. The per-class results from the classification reports explain this gap: Naive Bayes recorded a recall of 88% on the negative class but only 85% on the positive class, with precision following a similar pattern (86% negative, 88% positive), indicating a mild bias driven by its assumption that words occur independently of one another. Logistic Regression and Linear SVM produced more balanced results across both classes (90%/88% and 90%/89% precision for negative/positive respectively), which is consistent with their ability to weigh combinations of words more flexibly than the Naive Bayes independence assumption allows. This balance across both classes is the main reason the two linear models outperformed Naive Bayes on every metric.

Q4. Model Implementation for Clustering

Text clustering was applied to the same TF-IDF representation, but without ever using the sentiment label, to examine whether reviews form natural groupings based purely on word usage. Because the original TF-IDF matrix has 136,532 dimensions, it was first reduced to 2 dimensions using TruncatedSVD, the sparse-data equivalent of PCA. TruncatedSVD was chosen specifically because it operates directly on sparse matrices without requiring centering, which standard PCA requires and which would otherwise force the sparse TF-IDF matrix into a dense form too large to fit in memory. This single 2-dimensional reduction served both as the input to clustering and as the basis for visualization.

Selecting the Number of Clusters

Rather than assuming a cluster count, the number of clusters was determined using both the elbow method (within-cluster sum of squares) and the silhouette score, computed across $k = 2$ to 8 on a random subsample of 3,000 reviews for computational efficiency. The results are summarized in Table 4.1.

Table 4.1: WCSS and silhouette score across candidate values of k .

k	WCSS	Silhouette Score
2	12.80	0.4504
3	8.70	0.3833
4	6.83	0.3652
5	5.79	0.3659
6	5.01	0.3396
7	4.22	0.3325
8	3.81	0.3174

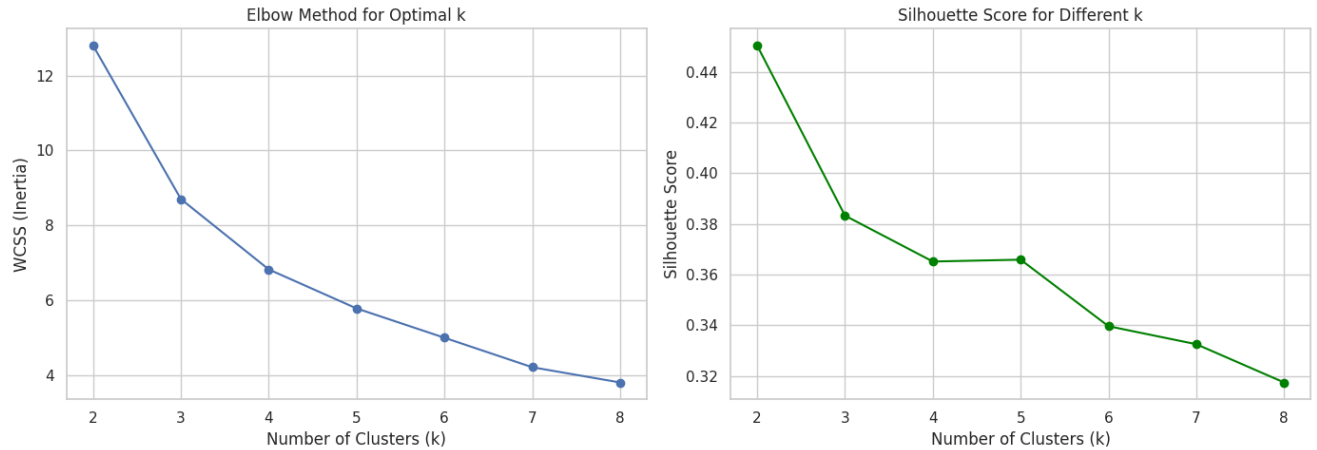


Figure 4.1: Elbow curve (WCSS) and silhouette score plotted side by side across $k = 2$ to 8.

The elbow curve did not show one sharply defined bend, but its steepest rate of decrease occurred between $k = 2$ and $k = 3$, and the silhouette score peaked clearly at $k = 2$ (0.4504), declining steadily for every larger value tested. Both pieces of evidence therefore supported $k = 2$ as the most well-separated and defensible choice, and this value was used consistently across all three clustering algorithms.

Clustering Algorithms and Results

Three clustering algorithms, each representing a different methodological family, were implemented and compared:

- KMeans — a partition-based algorithm that assigns each point to the nearest of k centroids.
- Agglomerative Clustering — a hierarchical algorithm that iteratively merges the closest pairs of points into larger groups.
- Gaussian Mixture Model (GMM) — a distribution-based algorithm that models each cluster as a probability distribution and assigns points based on the likelihood of membership, rather than a hard nearest-centroid assignment.

KMeans and GMM were fitted on the full reduced training set (39,665 reviews), since neither requires pairwise distance computation between every point. Agglomerative Clustering, however, does require pairwise distances and does not scale to the full dataset, so it was fitted on the same 3,000-review subsample used during k -selection. Cluster quality was evaluated using the silhouette score for all three algorithms, with results summarised in Table 4.2.

Table 4.2: Final cluster sizes and silhouette scores for the three clustering algorithms.

Algorithm	Cluster Sizes	Silhouette Score
KMeans	28,242 / 11,423	0.4359
Agglomerative	2,118 / 882	0.3873
Gaussian Mixture Model	28,481 / 11,184	0.4348

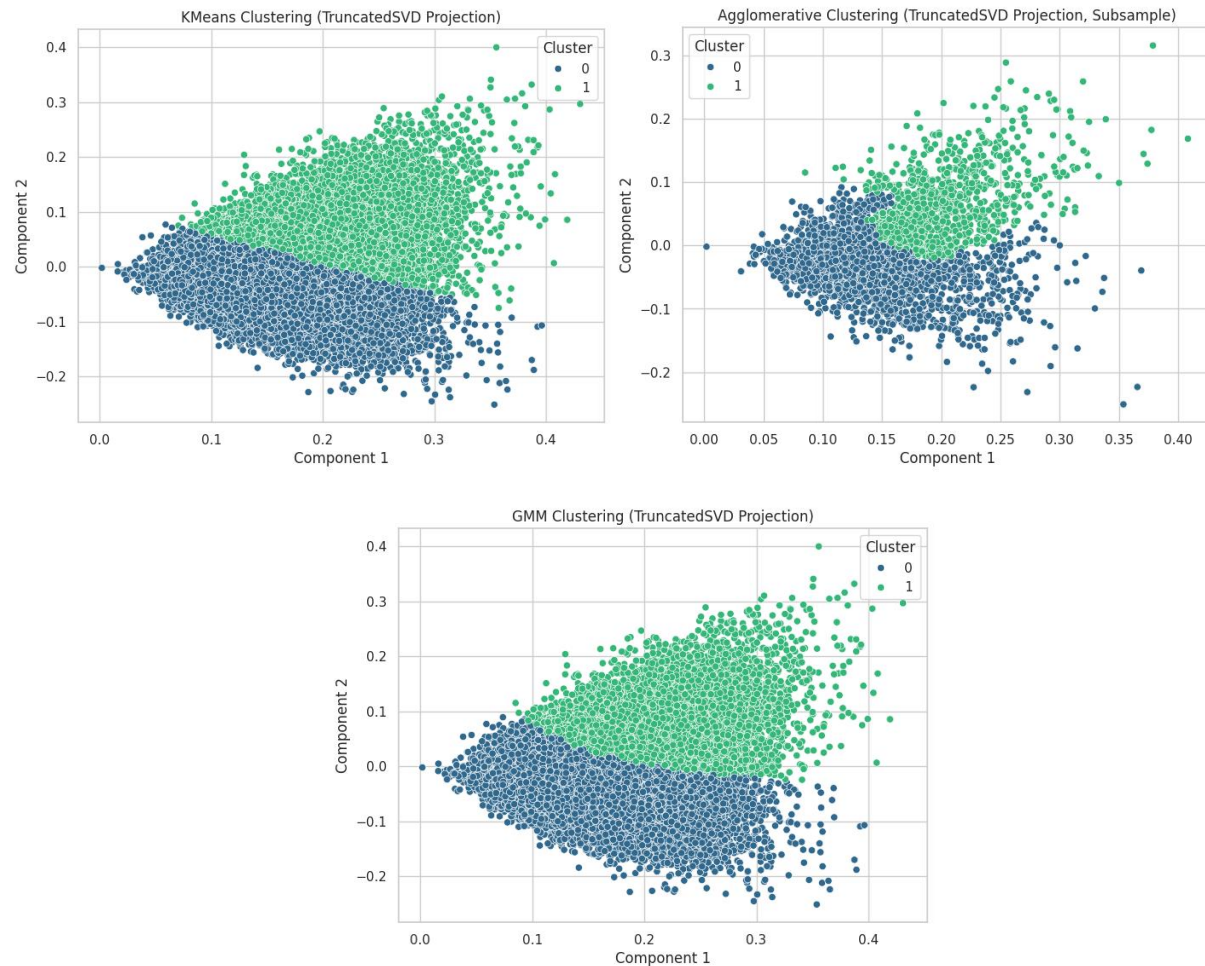


Figure 4.2: 2D scatterplots of the clusters produced by KMeans, Agglomerative Clustering, and the Gaussian Mixture Model.

KMeans achieved the highest silhouette score (0.4359), closely followed by GMM (0.4348); the two algorithms also produced almost identical cluster sizes, which is expected since both operate on the same reduced feature space and the same value of k . Agglomerative Clustering scored noticeably lower (0.3873), and produced a different split (roughly 71% to 29%) on its smaller subsample. Notably, the clusters formed by all three algorithms were imbalanced rather than

evenly split, in contrast to the dataset's near-balanced sentiment distribution (24,884 positive versus 24,698 negative). This suggests that the natural groupings discovered by clustering are not primarily driven by sentiment polarity, but more likely reflect other characteristics of the text such as review length, vocabulary diversity, or topic, since clustering had no access to the sentiment label at any point.

Q5. Sentiment Analysis

This section compares three fundamentally different approaches to sentiment analysis, all evaluated on the same test set of 9,917 reviews so that the comparison is fair across methods:

1. Lexicon-based (VADER): Valence Aware Dictionary and sentiment Reasoner relies on a pre-built dictionary of word sentiment scores and requires no training. It was applied directly to the raw review text, since it is specifically designed to interpret punctuation and capitalization as part of its scoring.
2. Machine learning (Logistic Regression): Rather than training a new model, the Logistic Regression classifier from Q3 was reused directly, since it was already trained on the full TF-IDF vocabulary (136,532 features) and had achieved strong results.
3. Deep learning (Long Short-Term Memory): A compact LSTM network was built using an Embedding layer followed by a single LSTM layer and a sigmoid output layer. Unlike the TF-IDF-based approaches, the LSTM processes review text as an ordered sequence of words, allowing it to capture word order and context, such as negation (e.g. “not good”).

Results

Table 5.1: Performance comparison of the lexicon-based, machine learning, and deep learning sentiment analysis approaches.

Approach	Accuracy	Precision	Recall	F1-Score
VADER (Lexicon)	0.6934	0.7151	0.6934	0.6851
Logistic Regression (ML)	0.8925	0.8929	0.8925	0.8925
LSTM (Deep Learning)	0.8764	0.8770	0.8764	0.8763

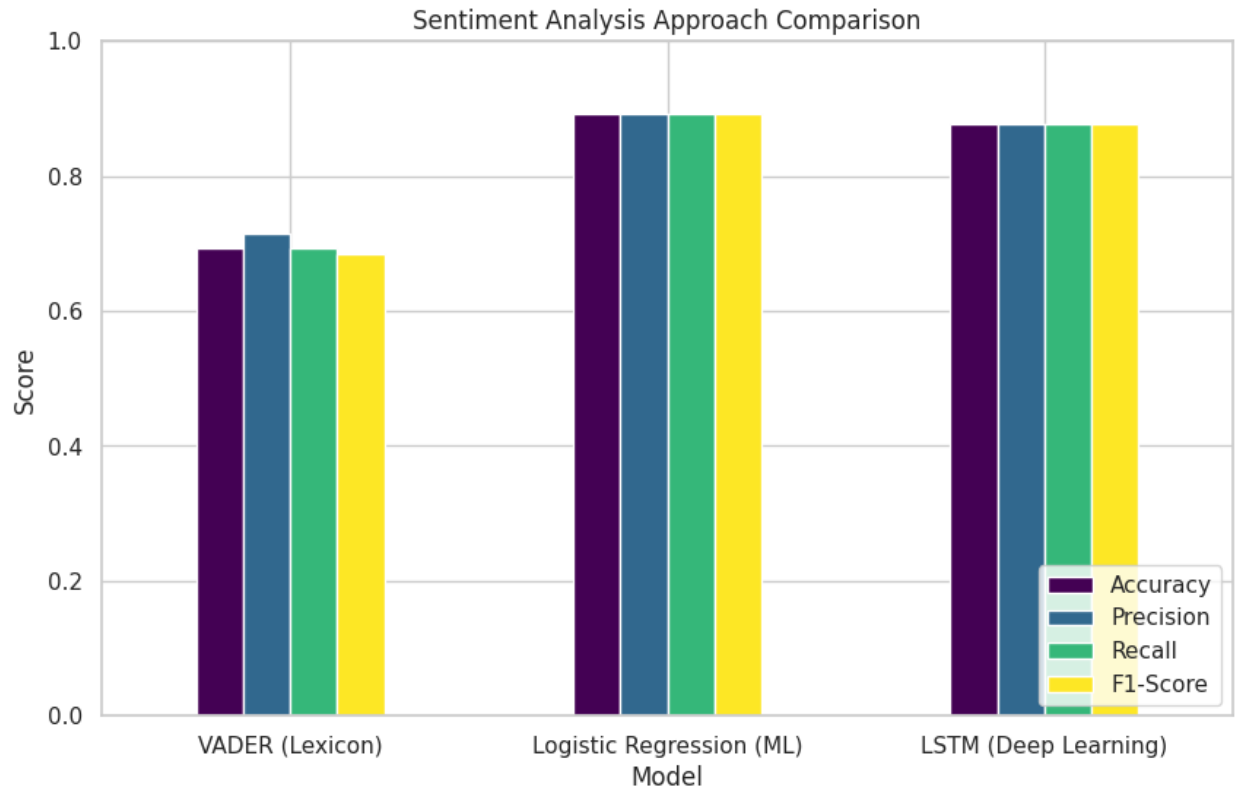
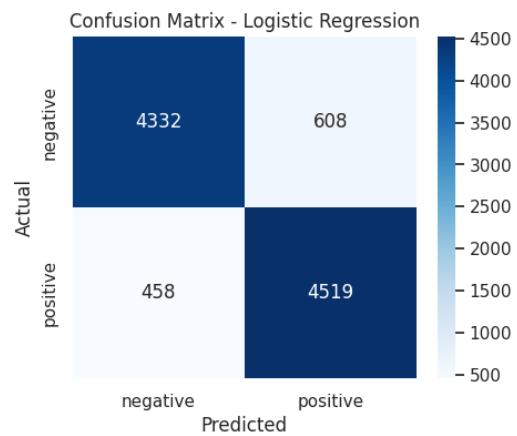
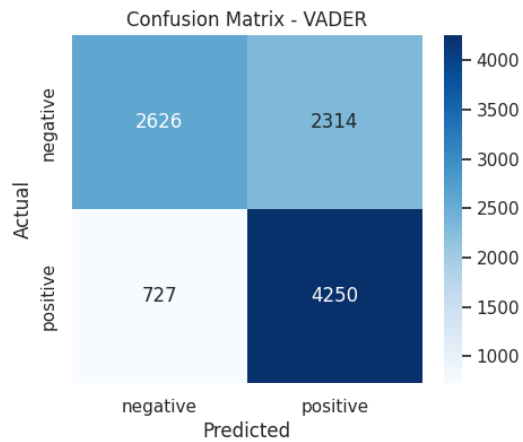


Figure 5.1: Bar chart comparing accuracy, precision, recall, and F1-score across the three sentiment analysis approaches.



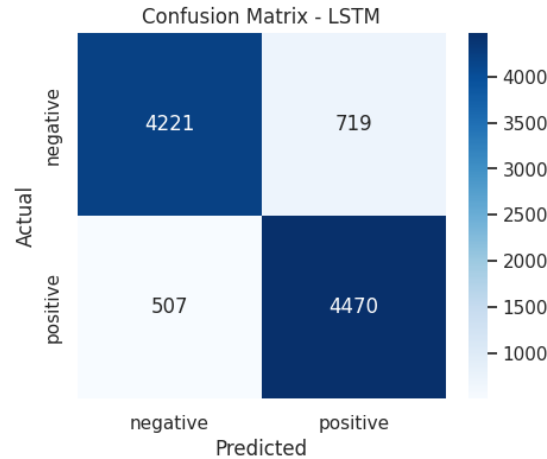


Figure 5.2: Confusion matrices for VADER, Logistic Regression, and the LSTM model.

VADER recorded the lowest accuracy of the three approaches (69.34%), reflecting the limitation of a fixed sentiment dictionary when applied to movie reviews, which often contain nuanced or sarcastic language that does not map cleanly onto individual word scores. Logistic Regression achieved the strongest result overall (89.25%), benefiting from access to the full 136,532-word TF-IDF vocabulary, which gives it precise, learned information about which exact words are associated with each sentiment. The LSTM achieved a competitive 87.64%, despite working with a smaller 10,000-word vocabulary and a fixed 200-word input window.

During development, the LSTM initially showed clear signs of overfitting: training accuracy climbed to approximately 98% while validation accuracy plateaued and declined to around 85%. This was addressed by introducing early stopping, configured to monitor validation accuracy with a patience of two epochs and to restore the best-performing weights rather than the final epoch's weights. This adjustment improved the LSTM's test accuracy from an initial 82.5% to 87.64%. A further refinement was also required: GPU-based LSTM training using cuDNN is non-deterministic by default, meaning that results varied slightly between repeated runs of identical code. This was resolved by explicitly seeding all relevant random number generators (Python, NumPy, and TensorFlow) and enabling TensorFlow's deterministic operation mode, after which the LSTM result became fully reproducible across repeated runs.

Pros and Cons of Each Approach

Table 5.2: Summary of the advantages and limitations of each sentiment analysis approach.

Approach	Pros	Cons
VADER (Lexicon)	No training required, instant to apply, fully interpretable	Ignores learned context and patterns; struggles with nuance and sarcasm
Logistic Regression (ML)	High accuracy, fast to train, interpretable feature weights	Treats words independently; ignores word order
LSTM (Deep Learning)	Captures word order and context; learns its own word representations	Requires more setup (tokenization, padding) and longer training time; prone to overfitting without regularization

Q6. Comparative Report and Discussion

This section consolidates the results of the previous three sections to compare model performance across all three NLP tasks, identify the strongest-performing approach in each case, and discuss the challenges encountered and the practical suitability of each method.

Performance Summary Across Tasks

Table 6.1: Best-performing model for each of the three NLP tasks.

Task	Best Model	Best Score
Text Classification (Q3)	Linear SVM	Accuracy = 0.8954
Clustering (Q4)	KMeans	Silhouette = 0.4359
Sentiment Analysis (Q5)	Logistic Regression	Accuracy = 0.8925

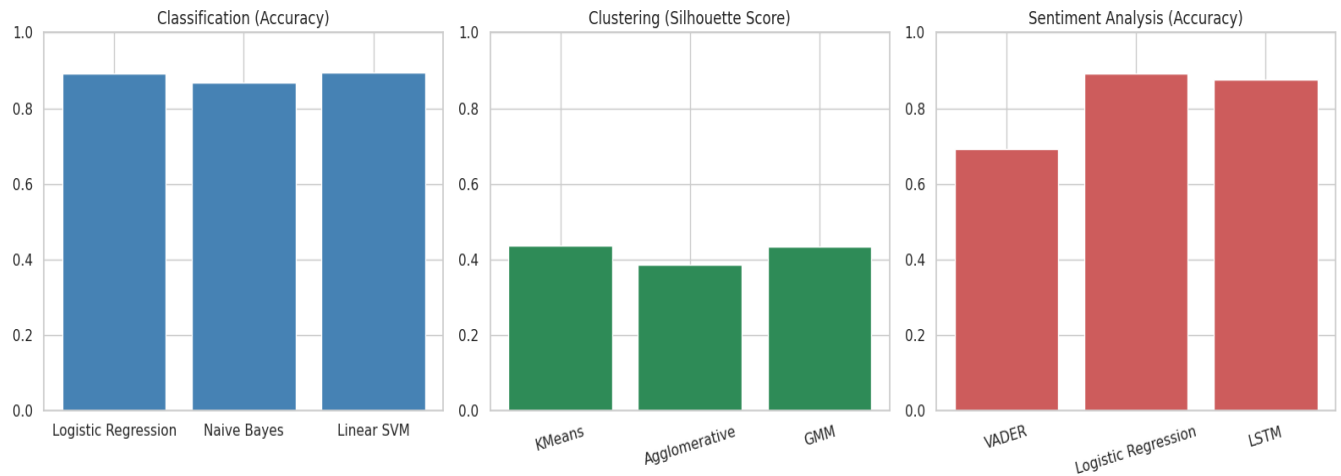


Figure 6.1: Combined comparison of classification accuracy, clustering silhouette score, and sentiment analysis accuracy across all models.

Which Models Performed Best and Why

In text classification, Linear SVM achieved the highest accuracy (89.54%), slightly ahead of Logistic Regression (89.25%), with both linear models clearly outperforming Naive Bayes (86.66%). This is consistent with the well-documented strength of margin-based and linearly-weighted models on high-dimensional sparse TF-IDF data, compared to Naive Bayes' simplifying assumption that words occur independently.

In clustering, KMeans and GMM produced nearly identical, well-separated results (silhouette scores of 0.4359 and 0.4348 respectively), both clearly ahead of Agglomerative Clustering (0.3873). However, as discussed in Q4, the clusters did not align closely with the dataset's balanced

sentiment distribution, indicating that sentiment is a difficult signal to recover from word usage alone without supervision.

In sentiment analysis, Logistic Regression was the strongest performer (89.25%), narrowly ahead of the LSTM (87.64%) and well ahead of VADER (69.34%). This result highlights that, on a dataset of this size with a rich enough vocabulary, a well-tuned linear model using the full TF-IDF feature space can match or exceed a deep learning model that must learn its own representations from a constrained vocabulary and a fixed-length input.

Challenges Faced During Preprocessing and Model Training

- Raw reviews contained leftover HTML tags (

), discovered during EDA as one of the most frequent “words” in the raw text (100,211 occurrences) — this required an explicit HTML-removal step before tokenization, since punctuation removal alone would not have eliminated it.
- TF-IDF vectorization produced 136,532 features, a dimensionality too high for direct use in clustering or in a dense neural network; TruncatedSVD was used to reduce the feature space to a manageable size suitable for both clustering and visualization.
- Agglomerative Clustering does not scale to the full 39,665-review training set due to its pairwise distance computation, requiring it to be evaluated on a 3,000-review subsample, while KMeans and GMM were both able to run on the full dataset without modification.
- The LSTM model initially overfit the training data (training accuracy of approximately 98% against a validation accuracy of approximately 85%); this was resolved by introducing early stopping with restoration of the best validation-performing weights, which raised test accuracy from 82.5% to 87.64%.
- LSTM results were initially inconsistent across repeated runs due to non-deterministic GPU operations in cuDNN; this was resolved by explicitly seeding all random number generators and enabling TensorFlow's deterministic operation mode, after which results became fully reproducible.

Scalability and Suitability in Real-World Applications

- Naive Bayes and Logistic Regression are lightweight and fast to train even on large vocabularies, making them well-suited to production systems that require frequent retraining or operate under limited computational resources.
- Linear SVM offers slightly higher accuracy than Logistic Regression but is comparatively slower to train as the dataset grows, which is a relevant trade-off at larger scale.
- KMeans and GMM both scale comfortably to large datasets and produced nearly identical results in this assignment; Agglomerative Clustering does not scale in the same way, due to its pairwise distance computation, and is better suited to smaller datasets or sampled subsets of larger ones.

- VADER requires no training data and produces results instantly, making it appropriate for quick, low-resource sentiment checks, though its accuracy is insufficient for applications where precision matters.
- The LSTM captures context and word order more effectively than TF-IDF-based models, but requires considerably more data, GPU resources, and careful tuning to reach its full potential, making it best suited to applications where context-sensitive language (such as sarcasm or negation) is common and the necessary computational resources are available.